

CocinaP

Intelligent Kitchen Security System

Technical & Creative Report

Version 1.0.1

July 2026

1. Executive Summary

CocinaP is a kitchen security system that uses artificial intelligence to detect and alert about hazardous situations in real time. It employs the YOLO11n (You Only Look Once) model to detect fire, smoke, people, and objects on the stove, continuously analyzing risk levels.

The system consists of two main components: a native Windows application built with Python and PySide6 that acts as a local server, and a mobile companion app for Android (with iOS support planned) developed in Flutter that enables remote monitoring over WiFi. Both applications communicate through a REST API, a real-time MJPEG video stream, and Server-Sent Events (SSE).

The primary goal is to reduce kitchen fire risk through early detection of fire, smoke, and unattended cooking situations, issuing local audible alerts and optional push notifications via Firebase Cloud Messaging (FCM).

2. What the Program Does

CocinaP functions as an automatic kitchen watchdog. It uses the computer's webcam to capture real-time video and processes it through a computer vision model (YOLO11n) that identifies the following elements:

2.1 Main Detections

- Fire: Detects flames and fire regions in the kitchen area, calculating fire coverage over the visible surface.
- Smoke: Identifies smoke presence, measuring its concentration and coverage in the scene.
- People: Recognizes human presence in the kitchen to determine if someone is supervising.
- Pots on stove: Detects pots, pans, and other objects over the stove area to identify unattended cooking.
- Stove zone: Users can configure a specific area in the image corresponding to the stove, enabling contextual detections.

2.2 Risk Analysis

The risk analyzer processes YOLO detections and classifies them into four severity levels:

- CRITICAL: Active fire or thick smoke with high coverage.
- HIGH: Significant smoke or incipient fire near the stove.
- MEDIUM: Unattended cooking for an extended period or light smoke.
- LOW: Very faint smoke or isolated detections.

2.3 System Actions

- Local audible alarm: The system emits beeps on the PC using Windows' Beep function, with distinct patterns for fire, smoke, and unattended cooking.
- Push notifications (FCM): If configured, sends push notifications to registered Android devices via Firebase Cloud Messaging.
- Real-time updates: The embedded web server transmits system status via SSE (Server-Sent Events) and video via MJPEG stream.

2.4 Native GUI (Windows)

The native Windows application presents three main tabs:

- Camera: Shows live video with overlaid detections, current risk status, and system FPS.
- Configuration: Allows adjusting detector parameters (confidence, IoU, stove area) and analyzer settings (timing, thresholds, FCM).
- Alarms: Event history and alerts generated by the system.

2.5 Mobile App (Android + iOS planned)

The companion mobile app enables:

- Automatic server discovery via mDNS/Zeroconf on the local network.
- Live video stream visualization.
- Status cards with fire, smoke, people, and pot counts.

- Remote configuration of system parameters.
- Alarm history with severity and type details.
- Foreground service that keeps the app active to receive alarm notifications even in background.

3. Technical Requirements

3.1 Hardware Requirements

System operation requires:

- Computer with Windows 10/11, 64-bit processor, minimum 8 GB RAM (16 GB recommended for optimal performance).
- Webcam (built-in or USB) with minimum 640x480 resolution, 1280x720 recommended.
- Storage: 1.5 GB of available space for the complete installation.
- Local WiFi connection (2.4 GHz or 5 GHz) for mobile device communication.
- Android 8.0+ (API 26+) device or iOS 15+ device (iOS support in development).

3.2 Software Dependencies (Backend - Windows)

The backend server is developed in Python 3.14.6 and requires:

- Python 3.14.6 (CPU-only, no CUDA)
- OpenCV (opencv-python-headless) - Image and video processing
- PySide6 - Native Windows GUI
- Ultralytics (YOLO11n) - Object detection model
- Pillow (PIL) - Additional image processing
- matplotlib - Internal visualizations
- sympy - Ultralytics math dependency
- zeroconf - mDNS network discovery
- firebase-admin (optional) - FCM push notifications
- numpy - Numerical computing
- pyinstaller (build) - Executable packaging

3.3 Software Dependencies (Mobile - Android & iOS)

The mobile app is developed in Flutter 3.x with Dart 3.x and targets both platforms:

- Flutter SDK 3.x (stable channel) - Cross-platform framework
- Dart SDK 3.x
- Android SDK 36, NDK 28, build-tools 36.0.0, JDK 21 (Android builds)
- Xcode 15+ (iOS builds on macOS)
- CocoaPods (iOS dependency manager)
- Key libraries: http, provider, shared_preferences, multicast_dns
- Firebase Core, Firebase Messaging (push notifications)
- flutter_local_notifications (local Android notifications)
- flutter_background_service (foreground service for Android)

3.4 iOS-Specific Requirements

To support iPhone, the following are needed:

- A Mac computer with macOS Ventura+ and Xcode 15+ to compile iOS builds.
- An Apple Developer account (\$99/year) to sign and distribute the app.
- APNs (Apple Push Notification service) configuration for push notifications.
- iOS 15.0 as minimum deployment target for compatibility.
- Info.plist additions: background modes for foreground service equivalent, camera permissions if needed, local network usage description for mDNS discovery.
- Potential migration from flutter_background_service to a native iOS Background Task implementation, since iOS background execution is more restrictive.
- TestFlight or App Store for distribution to testers and users.

3.5 Network & Connectivity

- Protocols: HTTP (REST API), MJPEG (video stream), SSE (real-time events), mDNS (automatic discovery)
- Default port: 8080 (configurable)
- Discovery: Zeroconf with service advertised as _cocinap._tcp.local.
- Firewall: Requires inbound rule for port 8080 on Windows Firewall
- iOS note: mDNS/Bonjour works on iOS natively, but requires NSLocalNetworkUsageDescription in Info.plist

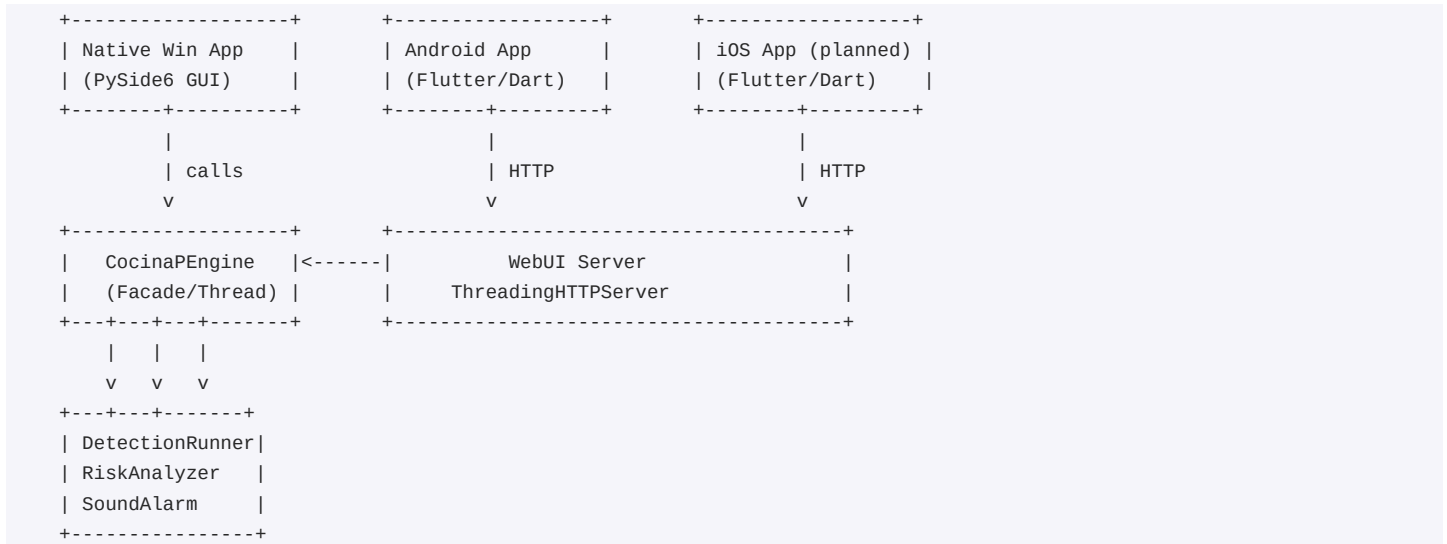
3.6 Installation & Distribution Requirements

- Windows: Inno Setup installer (~92 MB) deploying the PyInstaller-compiled executable, including the YOLO model and all dependencies.
- Android: APK/AAB for distribution (~50-150 MB depending on build type).
- iOS: IPA built via Xcode, distributed through TestFlight or App Store.
- Firebase: google-services.json (Android) / GoogleService-Info.plist (iOS) required only if FCM is desired.

4. System Architecture

4.1 Component Diagram

The system follows a modular architecture with a shared core (engine) that serves both the native GUI and the embedded web server:



4.2 Data Flow

1. The camera captures frames at full resolution (e.g. 1280x720).
2. DetectionRunner receives the frame via submit_frame() or callback.
3. The YOLO11n detector processes the frame at reduced resolution (320x320) and returns detections.
4. Detections pass to RiskAnalyzer which evaluates severity and determines whether to trigger an alarm.
5. SoundAlarm emits beeps if an active alert is present.
6. The frame with drawn detections is displayed on the native GUI.
7. WebUI serves the frame (no overlays, downscaled to 480px) via MJPEG for the mobile app.
8. WebUI publishes status via SSE and stores alarm history.
9. WebUI sends FCM notifications for critical/high alerts.

4.3 System Modules

- cocinap/config.py: Persistent JSON configuration management.
- cocinap/engine.py: System facade, orchestrates all modules.
- cocinap/detector/: YOLO detector and DetectionRunner (background thread).
- cocinap/analyzer/: RiskAnalyzer evaluating context and severity.
- cocinap/alarm/: SoundAlarm with async beeps via kernel32.Beep.
- cocinap/utils/: Drawing functions (draw_detections) and visualization.
- cocinap/webui.py: Embedded HTTP server (API, MJPEG, SSE, FCM, mDNS).
- cocinap/app.py: PySide6 app with 3 tabs (Camera, Config, Alarms).

4.4 REST API (Endpoints)

```
GET / - Basic server information
GET /api/stream - MJPEG live video stream
GET /api/events - Server-Sent Events (real-time alarms)
GET /api/status - Current status (detections, alerts)
GET /api/config - Current system configuration
GET /api/alarms - Alarm history (last 50)
GET /api/info - Server info (version, uptime, IPs)
POST /api/config - Update configuration
POST /api/register_token - Register FCM token
POST /api/unregister_token - Unregister FCM token
```

5. Creative Requirements

5.1 Native GUI Design (Windows)

The native GUI is built with PySide6 and follows these design principles:

- Visual clarity: The camera tab must display live video prominently, with minimal yet informative overlays (color-coded bounding boxes: red for fire, orange for smoke, green for people).
- Information hierarchy: Risk status must be immediately visible, with a color indicator (green = safe, yellow = attention, orange = caution, red = danger) and descriptive text.
- Immediate feedback: Any configuration change must reflect instantly, with visual indicators confirming the change was applied.
- Consistency: Standard Windows icons, system fonts, and familiar interaction patterns.
- Dark mode compatible: Color palette must work correctly under Windows dark theme.

5.2 Mobile App Design (Android & iOS)

The companion mobile app follows Material Design 3 with platform-specific adaptations:

- Discovery screen: A clean interface showing available servers on the local network with a visual connection indicator. The user must quickly identify their CocinaP server.
- Dashboard - Live Camera: The video stream should occupy most of the screen. Status cards below the video display fire, smoke, people, and pot counts with intuitive icons and traffic-light colors.
- Alarms screen: Reverse chronological list with color coding by severity. Each alarm shows type, severity, message, and timestamp.
- Configuration screen: Form with sliders and numeric fields organized by section, with a save button that sends changes to the server.
- Settings screen: Local device options such as refresh interval, notification preferences, and connection settings.

5.3 iOS Design Considerations

Adapting the app for iOS introduces additional design considerations:

- Navigation: Use iOS native tab bar (UITabBar) instead of Material tabs for a platform-consistent experience on iPhone.
- Typography: Use SF Pro font on iOS instead of Roboto.
- Gestures: Support iOS standard gestures (swipe back, pull-to-refresh).
- Status bar: Adapt to iPhone notch and Dynamic Island layout.
- Dark mode: Support iOS system-wide dark mode toggle via trait collections.
- Haptic feedback: Use UIImpactFeedbackGenerator for alarm notifications.
- Background execution: iOS has stricter background limits; the app should use BGTaskScheduler for periodic server health checks instead of a persistent foreground service.

5.4 User Experience (UX)

- Response time: The system should display video on the mobile app with a maximum latency of 2-3 seconds from camera capture.

- Notifications: Critical alerts must generate notifications with vibration and distinctive sound on both Android and iOS devices.
- Auto-reconnection: If WiFi is lost, the mobile app should attempt automatic reconnection to the server.
- Intuitive configuration: Technical parameters (YOLO confidence, IoU) should be presented with clear descriptions and predefined ranges preventing invalid values.
- Connection status: The top bar should show connection status with the server (connected/disconnected/searching).

5.5 Visual Identity

CocinaP's visual identity is based on the following elements:

- Name: CocinaP (Cocina + Prevention/Security).
- Color palette: Corporate blue (#193C78) as primary, with red (#DC3545) for danger, orange (#FD7E14) for warning, green (#28A745) for safe.
- Iconography: Kitchen and security icons (flame, smoke, person, pot, shield) using the Material Icons library, with SF Symbols equivalents for iOS.
- Communication tone: Professional and direct, with minimal technical jargon explained for non-expert users.

6. iOS Implementation Plan

Bringing CocinaP to iPhone requires addressing several platform-specific challenges. This section outlines the migration path from the current Android-only codebase to a fully cross-platform solution.

6.1 Platform-Specific Challenges

- Background execution: iOS does not allow persistent background services like Android. The BGTaskScheduler API must be used for periodic fetch tasks. The app can register for silent push notifications via FCM/APNs to wake up on alarm events.
- mDNS discovery: iOS supports Bonjour (the Apple implementation of mDNS) via NWConnection or the Flutter multicast_dns package. Requires NSLocalNetworkUsageDescription in Info.plist with user-facing permission dialog.
- Push notifications: FCM on iOS requires APNs (Apple Push Notification service) certificate/key configured in the Firebase Console, plus proper .entitlements file.
- MJPEG parsing: Same Dart code works cross-platform, no changes needed.
- Camera access: iOS requires NSCameraUsageDescription for any camera operations on the device (the iOS app will primarily view the server's camera, not use the device camera, so this may not apply).

6.2 Required Files & Configuration for iOS

- GoogleService-Info.plist from Firebase Console (iOS app registration).
- Info.plist additions: NSLocalNetworkUsageDescription, UIBackgroundModes (remote-notification, fetch).
- .entitlements for push notifications and APS environment.
- Podfile updates in ios/ directory for Flutter plugins.
- Xcode project configuration with proper bundle identifier, signing team, and deployment target (iOS 15.0+).

6.3 Build & Distribution Pipeline

- Development: flutter build ios --debug (runs on simulator or tethered device).
- TestFlight: Archive via Xcode, upload to App Store Connect for beta testing with up to 10,000 external testers.
- App Store: Production release requires Apple Developer Program (\$99/year).
- CI/CD: GitHub Actions with macOS runner can automate builds, signing, and TestFlight upload.

6.4 Estimated Effort

The iOS migration is estimated to require:

- 1-2 days: Flutter project configuration (ios/ folder setup, Podfile, Info.plist, Firebase config, code signing).
- 2-3 days: Background execution redesign (replace Android foreground service with iOS BGTaskScheduler + silent push wake-up).
- 1 day: mDNS permission handling and iOS-specific UX adjustments.
- 1 day: Testing on real devices, TestFlight submission, App Store preparations.
- Total: approximately 1 week for a complete iOS implementation.

7. Security & Privacy

CocinaP operates exclusively on the local network, without sending data to external servers. Security considerations include:

- All video processing is performed locally on the user's computer. No images are transmitted to the cloud or third parties.
- Communication between the mobile app and server occurs via HTTP on the local network (WiFi). There is no Internet exposure.
- The embedded HTTP server only listens on the local network interface (0.0.0.0:8080), accessible only from devices on the same subnet.
- mDNS discovery only works within the local network.
- Firebase Cloud Messaging is optional and only used for push notifications. No video data or personal information is sent to Firebase.
- No user registration, account, or authentication is required to operate.
- iOS note: Apple's App Review will require justification for `NSLocalNetworkUsageDescription`. The app must clearly explain why local network access is needed (server discovery).

8. Distribution Requirements

8.1 Windows Component

- Operating system: Windows 10/11 64-bit.
- No Python installation required (standalone executable compiled with PyInstaller).
- Installer: CocinaP_Setup_v1.0.1.exe (~92 MB) generated with Inno Setup.
- The installer deploys the executable, YOLO model, and all required DLLs.
- Disk space: ~850 MB for the complete installation (dist).
- Permissions: Administrator privileges required for installation.

8.2 Android Component

- Android 8.0+ (API 26+) device.
- APK signed with debug or release keys.
- Required permissions: INTERNET, FOREGROUND_SERVICE, FOREGROUND_SERVICE_DATA_SYNC, POST_NOTIFICATIONS, WAKE_LOCK.
- Requires enabling 'Install from unknown sources' for APK installation outside Google Play.
- Optional: google-services.json from Firebase Console for FCM.

8.3 iOS Component

- iPhone with iOS 15.0+.
- Development: Install via Xcode or TestFlight.
- Production: App Store distribution (requires Apple Developer Program, \$99/year).
- Required Info.plist entries: NSLocalNetworkUsageDescription, UIBackgroundModes (remote-notification, fetch).
- Required: GoogleService-Info.plist from Firebase Console for FCM/APNs.
- Apple Developer Program for push notification entitlements.

8.4 Network

- Both devices (PC and mobile) must be on the same WiFi network.
- Port 8080 must be open in the Windows Firewall. If the firewall rule cannot be added, manual IP connection is required.
- For manual connection, the mobile device needs the PC's local IP address.
- iOS note: Bonjour/mDNS on iOS prompts the user for local network permission on first launch.

9. Conclusion

CocinaP represents an accessible and effective solution for home fire prevention, combining modern computer vision technology with user-centered design. Its modular architecture allows independent maintenance and evolution of each component.

By operating entirely on the local network and not relying on cloud services, CocinaP guarantees user privacy and offers reliable operation even without Internet connectivity. Push notifications via FCM are optional and represent the only component requiring external connectivity.

Extending CocinaP to iOS (iPhone) is a natural next step that will significantly expand its user base. The Flutter framework already provides a strong foundation for cross-platform support, and the main effort lies in adapting to iOS-specific background execution models, push notification infrastructure, and App Store distribution requirements.

Key areas for improvement identified include: optimizing the video stream for mobile devices (resolution reduction implemented), fixing the MJPEG parser in the Android/iOS app (corrected), resolving mDNS discovery issues on restrictive firewall networks, and implementing the iOS platform layer.

CocinaP is ready for beta distribution on Android, with iOS support planned as the next major development milestone. Real-user validation in home environments will provide valuable feedback for future improvements.